# SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

## COURSE: CSE 316
## OPERATING SYSTEM

*SUBMITTED TO: Dr. Faculty*

## TOPIC

## "Inter-Process Communication (IPC) Mechanisms in Operating Systems"

Submitted by:


| | |
|---|---|
| **A.** | Full name |
| | **Harshdeep Singh** |
| Reg No. | **12409113** |
| Roll No | 39 |

# Table of Contents

## 1. Introduction

Inter-Process Communication (IPC) is a fundamental concept in modern operating systems that enables processes to exchange data, coordinate their execution, and share resources. In any multiprogramming environment, independent processes often need to cooperate to accomplish complex tasks.

However, processes by default operate in isolated memory spaces, making direct communication impossible. The OS provides several IPC mechanisms to bridge this gap:

- Shared Memory
- Message Queues
- Pipes and FIFOs (Named Pipes)
- Semaphores
- Signals
- Sockets

Traditional approaches without IPC suffer from:

- Process isolation leading to communication barriers
- Inability to synchronize concurrent processes
- Data inconsistency in shared-resource scenarios
- Race conditions and deadlocks without synchronization

To address these issues, this project — IPCXplorer — implements and demonstrates all major IPC mechanisms through a unified simulation framework. It provides:

- Practical implementation of each IPC technique in C
- Comparative analysis of performance and use-case suitability
- Visual representation of data flow and synchronization
- Producer-Consumer and Reader-Writer problem demonstrations

The IPCXplorer project bridges theory and practice, making IPC concepts tangible for students and developers working on concurrent systems.

## 2. Objectives of the Project

The IPCXplorer project aims to implement, analyze, and compare all major IPC mechanisms in Linux/Unix-based systems. The primary objectives are:

1. Implement All Major IPC Mechanisms – Provide working C implementations of Shared Memory, Message Queues, Pipes, FIFOs, Semaphores, and Signals.
2. Demonstrate Synchronization – Show how semaphores and mutexes solve the Producer-Consumer and Reader-Writer problems correctly.
3. Compare Performance – Measure throughput, latency, and overhead of each IPC technique under different workloads.
4. Visualize Data Flow – Provide a frontend dashboard that animates process communication and resource sharing in real-time.
5. Support Educational Use – Design the system as a learning tool for OS lab students studying IPC in CSE 325.
6. Handle Edge Cases – Implement proper cleanup of IPC resources (semaphore release, shared memory detach, pipe close) to prevent resource leaks.
7. Modular Architecture – Each IPC mechanism is implemented as an independent module, allowing focused study and testing.

## 3. Scope of the Project

IPCXplorer is designed as a versatile educational and analytical platform for studying inter-process communication. The scope of this project includes:

### 1. IPC in Unix/Linux Environments

- Implements POSIX and System V IPC APIs natively on Linux.
- Covers both kernel-mediated (message queues, semaphores) and user-space (pipes, shared memory) communication.
- Demonstrates real OS calls: shmget, msgget, pipe, mkfifo, sem_open, kill.

### 2. Academic and Industrial Application

- Serves as a reference implementation for OS lab experiments.
- Applicable in real-world embedded systems, server design, and concurrent application development.
- Helps evaluate synchronization strategies for multi-process applications.

### 3. Classic Problem Demonstrations

- Producer-Consumer problem using Shared Memory + Semaphores.
- Reader-Writer problem using Message Queues.
- Parent-Child synchronization using Pipes and Signals.

### 4. Performance Benchmarking

- Measures data transfer rates for each IPC mechanism.
- Compares latency under high-frequency message passing.
- Generates reports on resource consumption per mechanism.

### 5. Scalability

- Supports simulation with 2 to 20 concurrent processes.
- Extendable to add new IPC mechanisms such as Sockets or D-Bus.

## 4. System Architecture

Below is the high-level architecture of the IPCXplorer system:

| User Interface (Terminal CLI + Web Dashboard) |
| --- |
| Process Manager (fork/exec, PID tracking, lifecycle control) |
| IPC Engine: Shared Memory \| Message Queue \| Pipe \| FIFO \| Semaphore \| Signal |
| Synchronization Layer (Semaphore, Mutex, Condition Variables) |
| Metrics Collector (Throughput, Latency, Context Switches) |
| Visualization Layer (Data Flow Diagrams, Process Timeline) |

## 5. Module-Wise Breakdown

IPCXplorer is divided into five core modules, each responsible for a specific IPC mechanism. The modular design ensures focused implementation, independent testing, and easy extension.

### 5.a. Shared Memory Module

Implements POSIX shared memory for fastest inter-process data exchange.

**Key File: shared_memory.c**

**Responsibilities:**

- Create and attach shared memory segments using shmget / shmat.
- Implement Producer-Consumer pattern with circular buffer.
- Use semaphores for mutual exclusion on the shared buffer.
- Detach and destroy segments on termination (shmdt, shmctl IPC_RMID).

### 5.b. Message Queue Module

Implements System V message queues for structured, typed message passing.

**Key File: message_queue.c**

**Responsibilities:**

- Create queues using msgget; send/receive via msgsnd / msgrcv.
- Support multiple message types (priority-based retrieval).
- Demonstrate Reader-Writer problem with message typing.
- Clean up queue with msgctl IPC_RMID.

### 5.c. Pipe & FIFO Module

Implements both anonymous pipes (parent-child) and named pipes (FIFOs) for unrelated processes.

**Key File: pipe_fifo.c**

**Responsibilities:**

- Create anonymous pipes using pipe(); fork child process.
- Create named FIFOs using mkfifo() for unrelated process communication.
- Implement unidirectional and bidirectional data transfer.
- Handle pipe closure and EOF detection correctly.

### 5.d. Semaphore Module

Implements POSIX and System V semaphores for process synchronization and mutual exclusion.

**Key File: semaphore.c**

**Responsibilities:**

- Create named semaphores with sem_open(); unnamed with sem_init().

- Implement P (wait) and V (signal) operations.
- Prevent race conditions in critical sections.
- Demonstrate binary (mutex) and counting semaphores.

### 5.e. Signal Handling Module

Implements UNIX signal-based inter-process notification and control.

**Key File: signal_handler.c**

**Responsibilities:**

- Register custom signal handlers using sigaction().
- Send signals between processes using kill() and raise().
- Handle SIGCHLD for child termination notification.
- Implement graceful shutdown on SIGTERM / SIGINT.

## 6. Functional Requirements

This section specifies the functional behavior the IPCXplorer system must provide.

### 6.a. User-Level Requirements

- Users can select any IPC mechanism to demonstrate from the CLI menu.
- Users can configure the number of producer/consumer processes.
- Users can specify buffer size, message size, and iteration count.
- Users can run all IPC demos sequentially or individually.
- Output must display process PIDs, data exchanged, and timing metrics.

### 6.b. System-Level Requirements

- System must correctly create, use, and destroy all IPC resources.
- No zombie processes; all children must be reaped properly.
- Semaphores must prevent race conditions — no data corruption.
- Pipe reads/writes must be atomic for messages <= PIPE_BUF (4096 bytes).
- Message queues must support at least 10 concurrent message types.

### 6.c. Safety & Cleanup Requirements

- All IPC resources (shared memory, semaphores, queues) must be released on exit.
- Signal handlers must restore default behavior after cleanup.
- System must handle SIGINT (Ctrl+C) gracefully without resource leaks.

### 6.d. Non-Functional Requirements

- Performance: Each IPC demo must complete within 5 seconds for 1000 messages.
- Portability: Code must compile on any POSIX-compliant Linux system (GCC).
- Reliability: No segmentation faults or undefined behavior under normal operation.
- Usability: Clear console output with process IDs and transfer status.

### 6.e. Test Cases

8. Basic Pipe Test: Parent writes 'Hello', child reads and prints — verifies unidirectional flow.
9. Shared Memory Race: Two writers access buffer simultaneously without semaphore — data corruption expected; with semaphore — correct.
10. Message Priority: Messages of type 2 retrieved before type 1 using negative mtype — verifies priority.
11. Signal Test: Parent sends SIGUSR1 to child; child increments counter and reports — verifies signal delivery.
12. Cleanup Test: Kill producer mid-run; verify semaphore and shared memory are released on SIGINT.

## 7. Functionalities

IPCXplorer provides a comprehensive suite of IPC demonstrations and analysis tools:

13. Shared Memory Communication – Fastest IPC; processes share a memory region. Implements Producer-Consumer with circular buffer and semaphore-based synchronization.
14. Message Queue Passing – Structured message exchange with type-based retrieval. Demonstrates priority messaging and the Reader-Writer pattern.
15. Pipe Communication – Anonymous pipes for parent-child data flow. Bidirectional communication using two pipes.
16. FIFO (Named Pipe) – Persistent named pipes on the filesystem. Allows communication between unrelated processes by name.
17. Semaphore Synchronization – Binary and counting semaphores for mutual exclusion. Prevents race conditions in the Producer-Consumer scenario.
18. Signal-Based Notification – Lightweight inter-process notifications. Custom handlers for SIGUSR1, SIGUSR2, SIGCHLD, and SIGTERM.
19. Performance Benchmarking – Measures throughput (messages/sec) and latency (microseconds) for each mechanism under configurable load.
20. Interactive CLI – Menu-driven interface to select, configure, and run individual IPC demos or full comparison suite.

## 8. Technology Used

IPCXplorer is built using standard systems programming tools on Linux.

### 8.a. Programming Language

**C (C11 Standard)**

- Used for all IPC implementations — direct access to POSIX and System V APIs.
- Chosen for low-level control over process creation, memory, and file descriptors.
- Compiled with GCC using: gcc -Wall -Wextra -pthread -lrt -o ipcxplorer main.c

### 8.b. Libraries and Tools

**System Libraries (POSIX / System V):**

- <sys/ipc.h>, <sys/shm.h> — Shared Memory
- <sys/msg.h> — Message Queues
- <unistd.h>, <fcntl.h> — Pipes and FIFOs
- <semaphore.h>, <sys/sem.h> — Semaphores
- <signal.h> — Signal Handling
- <pthread.h> — Thread synchronization primitives
- <time.h> — Performance timing with clock_gettime()

### 8.c. Other Tools

- GCC — GNU Compiler Collection for building C source files
- Makefile — Automated build system with targets per IPC module
- GDB — Debugging race conditions and signal behavior
- Valgrind — Memory leak detection and IPC resource verification
- GitHub — Version control and commit tracking
- VS Code — Primary development environment with C/C++ extension

### 8.d. Environment

- OS: Ubuntu 22.04 LTS / Kali Linux (VirtualBox)
- Kernel: Linux 5.15+ (POSIX-compliant)
- Architecture: x86_64

## 9. Flow Diagram

The following represents the general flow of an IPC operation in IPCXplorer:

| **START** |
| --- |
| User selects IPC mechanism from CLI |
| Process Manager forks Producer and Consumer processes |
| IPC resource created (shmget / msgget / pipe / mkfifo / sem_open) |
| Producer writes data; Consumer reads data |
| Synchronization enforced (semaphore wait/post) |
| Metrics Collector records throughput and latency |
| IPC resource destroyed; processes exit cleanly |
| Results displayed on CLI + Visualization Dashboard |
| **END** |

## 10. Revision Tracking on GitHub

**Repository Name:**

IPCXplorer-OS-IPC-Mechanisms

**GitHub Link:**

https://github.com/riteshghosh12409135/IPCXplorer-OS-IPC-Mechanisms

**Deployed Website Link:**

https://ipcxplorer.vercel.app/ (Demo Dashboard)

**Commit Log:**

| Commit | Message | Summary |
|---|---|---|
| **Commit 1** | chore: initialize project structure | Created folder layout with src/, include/, Makefile, README.md, and .gitignore. |
| **Commit 2** | feat: implement pipe and FIFO IPC | Added pipe_fifo.c with anonymous pipe (parent-child) and named FIFO demos. |
| **Commit 3** | feat: implement shared memory + semaphore | Added shared_memory.c with POSIX shm and Producer-Consumer using sem_wait/sem_post. |
| **Commit 4** | feat: implement System V message queue | Added message_queue.c with msgget, msgsnd, msgrcv and priority retrieval demo. |
| **Commit 5** | feat: add signal handling module | Added signal_handler.c with SIGUSR1, SIGCHLD, SIGTERM handlers and graceful cleanup. |
| **Commit 6** | feat: performance benchmarking module | Added metrics.c for throughput and latency measurement across all IPC mechanisms. |
| **Commit 7** | docs: update README and add appendix | Updated README with build instructions, added code snippets and architecture diagrams. |

# 11. Conclusion and Future Scope

## 11.a. Conclusion

IPCXplorer successfully demonstrates how modern operating systems enable processes to communicate, synchronize, and share data through a variety of IPC mechanisms. By implementing Shared Memory, Message Queues, Pipes, FIFOs, Semaphores, and Signals in a unified C-based framework, the project highlights the strengths and trade-offs of each approach.

Key takeaways from the project include:

- Shared Memory is the fastest IPC mechanism but requires careful synchronization.
- Message Queues provide structured, type-safe communication at moderate speed.
- Pipes are simple and effective for linear parent-child data flow.
- Semaphores are the cornerstone of process synchronization in concurrent systems.
- Signals offer lightweight, asynchronous notifications but carry minimal data.

Through IPCXplorer, the importance of proper resource cleanup, deadlock prevention, and synchronization design is demonstrated practically — making this project a strong educational foundation for OS-level concurrent programming.

## 11.b. Future Scope

21. Socket-Based IPC – Extend to network sockets for inter-machine communication, enabling distributed system simulations.
22. D-Bus Integration – Add D-Bus as a modern, high-level IPC mechanism used in Linux desktop environments.
23. Deadlock Detection – Implement a deadlock detection module that visualizes circular wait conditions between processes.
24. Multi-Core Support – Demonstrate NUMA-aware shared memory and core-affinity-based IPC optimization.
25. Real-Time Visualization – Build an animated web dashboard showing live message flow, semaphore states, and buffer levels.
26. Android/Embedded Porting – Port the framework to Android (Binder IPC) and embedded Linux for IoT device studies.
27. Machine Learning Workload – Use IPC mechanisms as the backbone for a multi-process ML pipeline (data preprocessing, training, inference).

## 12. References


- W. Richard Stevens, Bill Fenner, Andrew M. Rudoff – "UNIX Network Programming, Vol. 2: Interprocess Communications"
- Abraham Silberschatz, Peter B. Galvin – "Operating System Concepts" (10th Edition)
- The Linux man-pages: https://man7.org/linux/man-pages/
- POSIX Standard – IEEE Std 1003.1: https://pubs.opengroup.org/onlinepubs/9699919799/
- GeeksforGeeks – IPC Mechanisms: https://www.geeksforgeeks.org/inter-process-communication-ipc/
- GNU C Library Documentation: https://www.gnu.org/software/libc/manual/

# 13. Appendix

## Appendix A: AI-Generated Project Elaboration

IPCXplorer is designed around the five pillars of IPC in Unix/Linux systems. Each mechanism solves a different class of communication problem:

| Mechanism | Speed | Persistence | Best Use |
|---|---|---|---|
| Shared Memory | Very Fast | Until shmctl IPC_RMID | Bulk data transfer |
| Message Queue | Moderate | Until msgctl IPC_RMID | Typed structured messages |
| Pipe (anon) | Fast | Process lifetime | Parent-child flow |
| FIFO (named) | Fast | Until unlinked | Unrelated processes |
| Semaphore | N/A (sync) | Until sem_destroy | Mutual exclusion |
| Signal | Very Fast | Instant | Notifications / events |

## Appendix B: Problem Statement

Traditional processes operate in isolated memory spaces and have no built-in mechanism to exchange data or coordinate execution. This project addresses the need to implement, compare, and visualize all standard POSIX/System V IPC mechanisms in a single educational framework, enabling students to understand when and how to apply each technique in real operating system scenarios.

## Appendix C: Code Snippets

**Directory Structure:**

```
IPCXplorer/
├── src/
│   ├── main.c           # CLI menu and demo runner
│   ├── shared_memory.c  # POSIX shared memory + semaphore
│   ├── message_queue.c  # System V message queues
│   ├── pipe_fifo.c      # Anonymous pipe + FIFO
│   ├── semaphore.c      # POSIX semaphore demos
│   ├── signal_handler.c # Signal handling demos
│   └── metrics.c        # Performance benchmarking
├── include/
│   └── ipc_common.h     # Shared constants and structs
├── Makefile
└── README.md
```

**Shared Memory: Producer-Consumer (shared_memory.c excerpt)**

```
#include <sys/shm.h>
#include <semaphore.h>
```

```
#include <stdio.h>

#define SHM_SIZE 4096
#define SEM_NAME "/ipc_sem"

int producer() {
    int shmid = shmget(IPC_PRIVATE, SHM_SIZE, IPC_CREAT | 0666);
    char *shm = (char *)shmat(shmid, NULL, 0);
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0644, 1);
    sem_wait(sem);
    sprintf(shm, "Hello from Producer PID=%d", getpid());
    sem_post(sem);
    sem_close(sem);
    shmdt(shm);
    return shmid;
}
```

### Message Queue: Send and Receive (message_queue.c excerpt)

```
#include <sys/msg.h>
#include <string.h>

struct msg_buf { long mtype; char mtext[256]; };

void send_message(int msqid, long type, const char *text) {
    struct msg_buf msg;
    msg.mtype = type;
    strncpy(msg.mtext, text, 255);
    msgsnd(msqid, &msg, sizeof(msg.mtext), 0);
}

void recv_message(int msqid, long type) {
    struct msg_buf msg;
    msgrcv(msqid, &msg, sizeof(msg.mtext), type, 0);
    printf("Received [type=%ld]: %s\n", msg.mtype, msg.mtext);
}
```